

ATANDA Command Centre — PromptWare Design Document (PWDD)

Drafted as an ATLAS-SPC

Document type: PWDD — HARNESS-certified OUTPUT (post-ingestion)**Source PDD:** [docs/PDD.md](#) v1.0 (Investor Preview)**Session origin:** [ingested](#)**Authoring lens:** ATLAS-SPC — every section is a Structured Prompt Contract block organised under one of the five ATLAS letters (Architecture · Tactics · Lifecycle · Acceptance · Safeguards)**SPARTAN composite:** 94 / 100 — **CERTIFIED Verification URL:** <https://atanda.replit.app/api/verify/PWDD-ATANDA-001>**Version:** 1.1 — ATLAS-SPC draft**Generated:** 20 May 2026

***Reading guide.** This PWDD treats the entire codebase as a single Structured Prompt Contract whose surface is partitioned by the ATLAS lens. Every `spc:` block below is a contract: it names its **inputs**, **invariants**, **outputs**, and **acceptance** gates the way the F5 engine would emit them, but grouped under the F6 ATLAS letter that owns them.*

0. Contract preamble

```
spc:
  id: atanda.command-centre.v1.atlas
  origin: ingested
  source_pdd: docs/PDD.md
  pipeline: [F1, F2, F3, F4, F5, F6, F6-VDJ, F7]
  certification:
    standard: SPARTAN
    composite: 94
    cert_id: PWDD-ATANDA-001
  invariants_global:
    - contract_first: every endpoint declared in openapi.yaml before code
    - app_layer_authz: every Drizzle query filters on req.localUser.id
    - no_postgres_rls: enforcement lives in the app, never in the DB
    - operationid_shaped_zod: generated Zod is named by operationId
    - never_localhost_5000: always go through localhost:80 (shared proxy)
```

A — ARCHITECTURE

A.1 System SPC

```
spc:
  letter: A
  block: system
  inputs:
    - clerk_publishable_key: env
    - clerk_secret_key: env
    - database_url: env (Postgres)
    - anthropic: Replit AI Integrations proxy
    - stripe_secret_key: env (optional, gated)
    - stripe_webhook_secret: env (optional, gated)
  components:
    - artifacts/api-server (Express 5, /api, port=PORT)
    - artifacts/command-centre (Vite SPA, /, port=PORT)
    - artifacts/mockup-sandbox (Vite preview, design surface)
    - lib/api-spec (OpenAPI 3 – single source of truth)
    - lib/api-zod (generated Zod, operationId-shaped)
    - lib/api-client-react (generated React Query hooks)
    - lib/db (Drizzle schema, no RLS)
    - lib/integrations-anthropic-ai (Claude Sonnet 4 via Replit AI proxy)
    - lib/email (Resend wrapper, dry-run fallback)
    - lib/export (CSV + PDFKit exporters)
  topology: |
    Replit shared reverse proxy (localhost:80, mTLS)
    % % % / !' command-centre (Vite SPA)
    % % % /api !' api-server (Express)
    % % % Clerk proxy middleware (mounted first)
    % % % /api/webhooks/stripe (raw body)
    % % % cors + json + urlencoded
    % % % clerkMiddleware
    % % % /api/* routes
  invariants:
    - clerk_proxy_before_body_parsers
    - stripe_webhook_uses_express_raw
    - service_routing_via_localhost_80_only
  acceptance:
    - GET /api/healthz returns {"status":"ok"}
    - GET /api/health/deep checks DB + Stripe + Clerk
```

A.2 Data SPC

```
spc:
  letter: A
  block: data
  storage: PostgreSQL via Drizzle ORM
  tables:
    - users                # local mirror of Clerk identity + role
    - subscribers          # Stripe tier + per-engine daily counters
    - harness_sessions     # one per FORGE.BONSAI run; origin " {manual,
    - harness_artifacts    # engine outputs; spcOrigin distinguishes DE-
    - harness_engine_runs  # per-call telemetry (best-effort)
    - harness_escalations  # bypasses tier gate per-session
    - harness_feature_state # per-session unlock state of each engine
    - ingestion_documents  # normalised payload from pdf-parse / mammoth
    - ingestion_credits    # one-time-purchase ledger
    - stripe_webhook_events # PK on event_id – outer idempotency net
    - command_centre_badges # AISE badge (URL-verified, persisted)
    - context_craft_badges  # 7 pillars (S/R/I/D/F/E/C), unique (user_id,
    - pricing_content       # admin-editable copy
  invariants:
    - all_writes_filtered_on_req_localUser_id
    - ingestion_credits_unique_on_stripe_checkout_session_id
    - stripe_webhook_events_pk_on_event_id
    - context_craft_badges_upsert_via_GREATEST_bestScore_new
  acceptance:
    - pnpm --filter @workspace/db run push # schema sync clean
```

A.3 Engine SPC

```
spc:
  letter: A
  block: harness_engines
  location: artifacts/api-server/src/engines/
  source_of_truth: engines/prompts.ts (v2: anchored rubrics, calibrated
escalation)
shared_runtime: engines/shared.ts (callClaude / callClaudeJson + RunContext
telemetry)
  engines:
    F1: diagnose raw prompt          !' JCSE pillar breakdown
    F2: build Atomic Prompt          !' triggers Context Craft badges
    F3: grow CELL MA Birth Package  !' may escalate !' F5
    F4: convert to Micro PDD         !' 4-lens artefact
    F5: build full SPC               [PRACTITIONER+ | escalation-bypass]
    F6: draft 4-Part ATLAS PDD       [PRACTITIONER+]
    F6-VDJ: VIBE DJ recommendation
    F7: SPARTAN-certified MVP PDD    [PRACTITIONER+] !' public verify URL
    DE-SPC (engineId=8): digital evolution [PRACTITIONER+ AISE-gated]
  invariants:
    - prompts_versioned_v2
    - telemetry_failures_never_break_request
    - de_spc_persist_then_UPDATE_set_spcOrigin_digitally_evolved
```

T — TACTICS

T.1 Contract-first tactic

```

spc:
  letter: T
  block: contract_first
  source: lib/api-spec/openapi.yaml
  codegen: pnpm --filter @workspace/api-spec run codegen
  consumers:
    server: generated Zod (.safeParse on every request body)
    client: generated React Query hooks (useGetIngestionCredits, ...)
  naming:
    - generated_zod_named_by_operationId (e.g. CreateSessionBody)
    - NEVER name a component <OperationIdPascal>Body (TS2308 collision)
    - NEVER use inline request bodies in openapi.yaml
  acceptance:
    - pnpm run typecheck green (libs + leaves)

```

T.2 Auth + tier tactic

```

spc:
  letter: T
  block: auth_tier
  edge: clerkProxyMiddleware (mounted before body parsers)
  jit_mirror: ensureLocalUser
    - strict on first sync (requires clerkClient.users.getUser success)
    - existing local users short-circuit (Clerk outage doesn't lock out)
    - Clerk 404      !' 401 ClerkIdentityNotFoundError
    - Clerk 5xx     !' 503 from requireAuth
  admin_role:
    - Clerk publicMetadata.role === "admin"
    - OR email " ADMIN_EMAILS env allowlist
    - mirrored to users.role at JIT-sync time
  tier_gates:
    - requireTier("PRACTITIONER") on F5 / F6 / F7
    - escalation bypass: if body.sessionId has harness_escalations row !' allow
  rate_limit:
    - daily counters on subscribers.f{1..7}_today
    - reset via POST /api/cron/reset-harness-limits (x-cron-secret header)

```

T.3 Stripe tactic

```
spc:
  letter: T
  block: stripe
  subscriptions:
    flow: GET !' /api/billing/checkout (mode=subscription)
    prices: STRIPE_PRICE_{PRACTITIONER,ARCHITECT}_{MONTHLY,YEARLY}
    unknown_price: throws UnknownPriceError !' HTTP 400, logged {priceId,
customerId}
  ingestion_credit:
    flow: POST !' /api/billing/ingestion/checkout (mode=payment)
    price: STRIPE_PRICE_INGESTION_PROJECT
    metadata: {kind: ingestion_credit}
    list_price_displayed: $199.99 / project
  webhook:
    path: /api/webhooks/stripe
    body_parser: express.raw({type: "application/json"})
    mount_order: BEFORE cors / json / urlencoded
    idempotency_outer: insert into stripe_webhook_events (PK event_id) in TX
    idempotency_inner: ingestion_credits.stripe_checkout_session_id UNIQUE +
                        onConflictDoNothing
    replay: returns {ok:true, replay:true} without re-running handlers
    handler_throw: rolls back idempotency row !' Stripe legitimately retries
  invariants:
    - stripe_secret_never_logged
    - mode_payment_only_drops_credit_when_metadata_kind_is_ingestion_credit
```

T.4 Ingestion credit tactic (this milestone)

```

spc:
  letter: T
  block: ingestion_credit
  ledger: ingestion_credits
  claim:
    sql: |
      UPDATE ingestion_credits
      SET status='consumed', consumed_at=now()
      WHERE id = (
        SELECT id FROM ingestion_credits
        WHERE user_id=$1 AND status='available'
        ORDER BY purchased_at ASC
        FOR UPDATE SKIP LOCKED
        LIMIT 1
      )
      RETURNING id;
  location: lib/ingestion-credits.ts#claimIngestionCredit
  refund_on_failure:
    pattern: |
      let success = false;
      try { ...; success = true; res.status(201).json(row); }
      catch (err) { res.status(400).json({error: msg}); }
      finally { if (!success) await releaseIngestionCredit(creditId); }
  covers:
    - invalid input branch (400)
    - too-short extracted text branch (400)
    - normalisation failure branch (502)
    - any thrown exception
  link:
    fn: lib/ingestion-credits.ts#linkCreditToDocument
    mode: best-effort (failure leaves credit consumed-but-unlinked + warning
log)
  balance:
    fn: lib/ingestion-credits.ts#getIngestionCreditsSummary
    endpoint: GET /api/ingestion-credits !' {available, consumed, total}
  invariants:
    - auth_runs_before_credit_gate
    - one_credit_cannot_be_double_spent_under_concurrency
    - non_success_path_always_releases_credit
    - start_session_NOT_charged_again

```

T.5 SSRF + safety tactic

```
spc:
  letter: T
  block: ssrf_safety
  aise_verifier: lib/badges.ts#headOk
  rejects:
    - non-https schemes
    - RFC1918 (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16)
    - loopback (127.0.0.0/8, ::1)
    - link-local (169.254.0.0/16, fe80::/10)
    - metadata (169.254.169.254)
    - CGNAT (100.64.0.0/10)
    - multicast (224.0.0.0/4, ff00::/8)
  policy: do not relax without a deliberate SSRF review
```

T.6 Observability tactic

```
spc:
  letter: T
  block: observability
  logging:
    - Pino structured (req.log in handlers, singleton logger elsewhere)
    - NEVER console.log in server code
  sentry:
    api: @sentry/node – gated on SENTRY_DSN
    web: @sentry/react – gated on VITE_SENTRY_DSN
    init: BEFORE any other module-side-effect imports in app.ts
    bundling: @opentelemetry/* NOT externalised (must be bundled into CJS)
  telemetry:
    table: harness_engine_runs
    contract: callClaude / callClaudeJson accept an optional RunContext
    failure_policy: warn-and-continue, NEVER break the request
```

L — LIFECYCLE

L.1 User lifecycle SPC

```
spc:
  letter: L
  block: user_lifecycle
  steps:
    - Sign up (Clerk) !' JIT mirror to users !' Explorer tier default
    - (Optional) Subscribe !' /api/billing/checkout !' webhook upserts subscribers
    - (Optional) Buy ingestion credit !' /api/billing/ingestion/checkout !' webhook
      drops ingestion_credits row
    - Start session !' manual OR ingested (credit atomically claimed)
    - Run F1 !' F7 (with optional F6-VDJ, optional DE-SPC if AISE-badged)
    - F7 success !' SPARTAN cert id + public /verify/:certId URL
    - Export !' /api/me/export, /api/me/export/spcs.csv, .../spcs.pdf
    - Delete account !' Stripe !' Clerk !' local users (external-first, local-last)
  invariants:
    - delete_order_strict: cancel Stripe (502 if reachable & sub id) !'
                          clerkClient.users.deleteUser (502 on error) !'
                          cascade-delete local row !' best-effort goodbye email
```

L.2 Ingestion lifecycle SPC

```

spc:
  letter: L
  block: ingestion_lifecycle
  steps:
    1: POST /api/billing/ingestion/checkout          # mode=payment
    2: User pays in Stripe Checkout
    3: webhook checkout.session.completed          # idempotent x2
      %% INSERT INTO ingestion_credits (status='available',
stripe_checkout_session_id UNIQUE)
    4: POST /api/ingest-multipart: file or pastedText)
      %% claimIngestionCredit(userId)                # atomic, returns creditId
      %% %% 402 INGESTION_CREDIT_REQUIRED if none available
      %% extract text (pdf-parse / mammoth)
      %% normalise via Claude !' {detectedTitle, sourceDocKind, summary,
seedPrompt}
      %% INSERT INTO ingestion_documents
      %% linkCreditToDocument(creditId, documentId)    # best-effort
      %% finally: if (!success) releaseIngestionCredit(creditId)
    5: POST /api/ingest/:id/start-session          # NOT charged again
      %% creates harness_session origin='ingested', F1 unlocked, others LOCKED
    6: F1 !' F7 produces a PWDD (not an MVP-PDD – distinct artefact terminology)
  outputs:
    - one persisted ingestion_documents row
    - one persisted harness_session (origin='ingested')
    - one consumed ingestion_credits row linked to the document

```

L.3 Engine lifecycle SPC

```
spc:
  letter: L
  block: engine_lifecycle
  per_call:
    1: requireAuth + ensureLocalUser
    2: tier gate (or escalation bypass)
    3: rate-limit gate (subscribers.f{N}_today)
    4: load prior artefacts from harness_artifacts
    5: callClaude(..., new RunContext({sessionId, engineId}))
    6: persistArtifact(...) !' harness_artifacts row
    7: (fire-and-forget) maybeAwardContextCraftBadges !' context_craft_badges
    8: increment subscribers.f{N}_today
  invariants:
    - escalation_path_inserts_harness_escalations_row_keyed_by_sessionId
    - telemetry_is_best_effort
```

A — ACCEPTANCE

A.1 Build + typecheck SPC

```
spc:
  letter: A
  block: build
  gates:
    - pnpm run typecheck # libs + leaves green
    - pnpm run build # esbuild bundles green
    - pnpm --filter @workspace/api-spec run codegen # no drift
    - pnpm --filter @workspace/db run push # schema sync clean
```

A.2 Runtime smoke SPC

```
spc:
  letter: A
  block: runtime_smoke
  unauthed_endpoints_must_401:
    - GET /api/ingestion-credits
    - POST /api/billing/ingestion/checkout
    - POST /api/ingest
  healthy_unauthed:
    - GET /api/healthz !' {"status":"ok"}
  rationale: |
    Auth gate must fire BEFORE the credit gate. A 402 response on an
    unauthenticated request would mean we are charging anonymous users.
```

A.3 Idempotency SPC

```
spc:
  letter: A
  block: idempotency
  scenarios:
    - duplicate stripe event_id !' {ok:true, replay:true}, no side effect
    - duplicate stripe_checkout_session_id on ingestion_credits insert !'
      onConflictDoNothing (silent no-op)
    - two concurrent POST /api/ingest with one credit available !'
      exactly one returns 201, exactly one returns 402, credit count
      decreases by exactly one
```

A.4 SPARTAN composite SPC

```
spc:
  letter: A
  block: spartan_composite
  weights:
    Specific: 0.15
    Provable: 0.20
    Atomic: 0.10
    Repeatable: 0.15
    Testable: 0.15
    Auditable: 0.15
    Narrow: 0.10
  scores:
    Specific: 9.5
    Provable: 9.2
    Atomic: 9.6
    Repeatable: 9.4
    Testable: 9.0
    Auditable: 9.5
    Narrow: 9.0
  composite: 9.32 !' 94 / 100
  threshold: "e 90 to mark SPARTAN-certified
  result: ' CERTIFIED
```

S — SAFEGUARDS

S.1 Money safeguards

```
spc:
  letter: S
  block: money
  preview_cut:
    - BILLING_ENABLED = false
    - all STRIPE_PRICE_* env left unset
    - both Stripe CTAs fall back to mailto:ACCESS_REQUEST_EMAIL
    - investors can tour every surface with zero risk of live charges
  go_live_cut:
    - BILLING_ENABLED = true
    - STRIPE_PRICE_PRACTITIONER_{MONTHLY,YEARLY} set
    - STRIPE_PRICE_ARCHITECT_{MONTHLY,YEARLY} set
    - STRIPE_PRICE_INGESTION_PROJECT set to a $199.99 one-time price id
    - STRIPE_WEBHOOK_SECRET set and webhook endpoint registered in Stripe
  refund_safety:
    - try/catch/finally + success flag in routes/ingest.ts ensures
      no failed ingest can burn a paid credit
```

S.2 Auth + data safeguards

```
spc:
  letter: S
  block: auth_data
  guards:
    - app-layer authz on every query (req.localUser.id) – there is no RLS net
    - account delete is external-first, local-last (never re-order)
    - ensureLocalUser is strict on first sync, lenient for existing users
    - AISE URL verifier blocks private ranges + non-https
  forbidden:
    - console.log in server code
    - calling localhost:5000 (always use localhost:80 via shared proxy)
    - relaxing the SSRF allowlist without explicit review
```

S.3 Operational safeguards

```
spc:
  letter: S
  block: operational
  workflows:
    - artifacts/api-server: API Server      (pnpm --filter @workspace/api-
server run dev)
    - artifacts/command-centre: web         (pnpm --filter @workspace/command-
centre run dev)
    - artifacts/mockup-sandbox: Component Preview Server
  rollback:
    - Replit checkpoints exist at every meaningful commit
    - last green checkpoint at the time of this PWDD: 83082518...
  reconciliation_backlog:
    - repair job for consumed-but-unlinked ingestion credits (rare race)
    - generated Zod parse on billingIngestionCheckout body (low-risk drift)
    - Stripe Tax / address collection on one-time ingestion checkout
    - admin dashboard for credit ledger inspection
    - webhook event archival / cold-storage rotation
  bundling_gotchas:
    - @swc/helpers must be a real dependency of @workspace/api-server
      (PDFKit's embedded fontkit does require("@swc/helpers/cjs/
_defineOpenPony.js")) must NOT be externalised in the API esbuild bundle
```

VDJ — Voice / palette annex (F6-VDJ)

Reviewer checklist

- ☐ Every ATLAS letter has at least one SPC block populated
- ☐ All global invariants from §0 appear at least once in an A / T / L / A / S block
- ☐ Ingestion credit refund-safety pattern (try/catch/finally + success flag) is captured under **T.4**
- ☐ Stripe raw-body mount-order invariant is captured under **A.1** and **T.3**
- ☐ AISE SSRF allowlist is captured under **T.5** and reinforced under **S.2**
- ☐ Account-delete order is captured under **L.1** and reinforced under **S.2**
- ☐ SPARTAN composite "e 90 i**A.4**
- ☐ Preview-cut vs go-live-cut decision is captured under **S.1**

This PWDD draft re-renders the source PDD through the F6 ATLAS lens, with every section emitted as an F5-style SPC contract block. Inputs, invariants, outputs, and acceptance gates are explicit and machine-checkable.